

Programowanie obiektowe i inżynieria oprogramowania

Lab AG-8:

- lista inicjalizacyjna,
- konstruktor delegujący,
- konstruktor kopiujący.

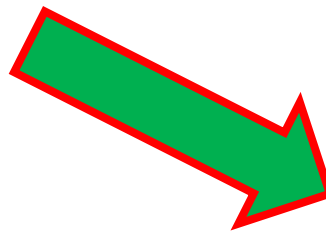
Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2025

Lista inicjalizująca

```
main.cpp  TCandidate.h  TPopulation.h
2024
(Globalny zasięg)
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4
5  // #include "TAlgorithm.h"
6  #include "TParam.h"
7
8  using namespace std;
9
10
11  int main()
12  {
13      srand(time(0));
14
15      TParam gen{ "gen1", 0, 10, 0.1 };
16      gen.info();
17
18      cout << "\n\n\n";
19      return 0;
20  }
```

Tworzymy obiekt klasy
TParam



```
Konsola debugowania programu Microsoft Visual Studio

=====
== name: gen1
== range: [0; 10; 0.1]
== value: 6.5      (id: #65)
=====
```

Zmieniamy parametr klasy *name* na stałą.

Komentujemy funkcję `set_name` – nie ma ona zastosowania dla wartości stałej.

```
TParam.h  X main.cpp  TCandidate.h  TPopulation.h  TAlgorithm.h
AG2024    TParam
1  #pragma once
2
3  #include <string>
4
5
6  class TParam
7  {
8  private:
9      const std::string name;
10     double x_start, x_end, dx;
11     int val_id;
12
13  public:
14     TParam(double x_start, double x_end, double dx);
15     TParam(double x_start, double x_end, double dx, double val);
16     TParam(std::string name, double x_start, double x_end, double dx);
17     TParam(std::string name, double x_start, double x_end, double dx, double val);
18
19     // void set_name(std::string name) { this->name = name; }
20     std::string get_name() { return name; }
```

Próba uruchomienia programu zwraca błędy:

```
✗ C2678  dwuargumentowy "=": nie znaleziono żadnego operatora, który przyjmuje lewostronny operand typu "const std::string" (lub nie istnieje akceptowalna konwersja)
✗ C2678  dwuargumentowy "=": nie znaleziono żadnego operatora, który przyjmuje lewostronny operand typu "const std::string" (lub nie istnieje akceptowalna konwersja)
✗ C2678  dwuargumentowy "=": nie znaleziono żadnego operatora, który przyjmuje lewostronny operand typu "const std::string" (lub nie istnieje akceptowalna konwersja)
✗ C2678  dwuargumentowy "=": nie znaleziono żadnego operatora, który przyjmuje lewostronny operand typu "const std::string" (lub nie istnieje akceptowalna konwersja)
```



```
TParam.cpp  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp
AG2024
1  #include <iostream>
2  #include <math.h> // fabs
3  #include <cstdlib> // rand
4
5  #include "TParam.h"
6
7  using namespace std;
8
9  TParam::TParam(double x_start, double x_end, double dx)
10 {
11     set_range(x_start, x_end, dx);
12     name = "";
13     set_rand_val();
14 }
15
16 TParam::TParam(double x_start, double x_end, double dx, double val)
17 {
18     set_range(x_start, x_end, dx);
19     set_val(val);
20     name = "";
21 }
```

Błędy przy kompilacji wynikają z próby przypisania wartości do stałej *name* w konstruktorze.

Z powyższego wynika, że obiekt i jego zmienne/stałe istnieją już w momencie pracy konstruktora.

Lista inicjalizująca



**Rozwiązaniem
problemu są
listy
inicjalizujące!**

```
TParam.cpp  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp
AG2024      ↓ TParam
1  #include <iostream>
2  #include <math.h> // fabs
3  #include <cstdlib> // rand
4
5  #include "TParam.h"
6
7  using namespace std;
8
9  TParam::TParam(double x_start, double x_end, double dx) : name{""}
10 {
11     set_range(x_start, x_end, dx);
12     // name = "";
13     set_rand_val();
14 }
15
16 TParam::TParam(double x_start, double x_end, double dx, double val) : name{ "" }
17 {
18     set_range(x_start, x_end, dx);
19     set_val(val);
20     // name = "";
21 }
22
23 TParam::TParam(string name_val, double x_start, double x_end, double dx) : name{name_val}
24 {
25     //this->name = name;
26     set_range(x_start, x_end, dx);
27     set_rand_val();
28 }
29
30 TParam::TParam(string name, double x_start, double x_end, double dx, double val) : name{name}
31 {
32     //this->name = name;
33     set_range(x_start, x_end, dx);
34     set_val(val);
35 }
```

```
Konsola debugowania programu Mic
=====
== name: gen1
== range: [0; 10; 0.1]
== value: 4.4      (id: #44)
=====
```

Konstruktor delegujący

Konstruktor delegujący



Konstruktor delegujący (linia 16 i 30) – część zadań jest wykonywana przez utworzony już wcześniej konstruktor. Uzupełnienie wartości części zmiennych obiektu jest realizowana przez utworzony wcześniej konstruktor. W nowym konstruktorze zajmujemy się jedynie obsługą nowych/kolejnych zmiennych.

```
TParam.cpp  TCandidate.cpp  TPopulation.cpp  T
AG2024
1  #include <iostream>
2  #include <math.h> // fabs
3  #include <cstdlib> // rand
4
5  #include "TParam.h"
6
7  using namespace std;
8
9  TParam::TParam(double x_start, double x_end, double dx) : name{""}
10 {
11     set_range(x_start, x_end, dx);
12     // name = "";
13     set_rand_val();
14 }
15
16 TParam::TParam(double x_start, double x_end, double dx, double val) : TParam(x_start, x_end, dx)
17 {
18     // set_range(x_start, x_end, dx);
19     set_val(val);
20     // name = "";
21 }
22
23 TParam::TParam(string name_val, double x_start, double x_end, double dx) : name{name_val}
24 {
25     //this->name = name;
26     set_range(x_start, x_end, dx);
27     set_rand_val();
28 }
29
30 TParam::TParam(string name, double x_start, double x_end, double dx, double val) : TParam(name, x_start, x_end, dx)
31 {
32     // this->name = name;
33     // set_range(x_start, x_end, dx);
34     set_val(val);
35 }
36
```


Konstruktor kopiujący

Konstruktor kopiujący



```
TParam.h  main.cpp  TCandidate.h  TPopulation.h  TAlgorithm.h
AG2024    (Globalny zasięg)

1  #pragma once
2
3  #include <string>
4
5
6  class TParam
7  {
8  private:
9      const std::string name;
10     double x_start, x_end, dx;
11     int val_id;
12
13 public:
14     TParam(double x_start, double x_end, double dx);
15     TParam(double x_start, double x_end, double dx, double val);
16     TParam(std::string name, double x_start, double x_end, double dx);
17     TParam(std::string name, double x_start, double x_end, double dx, double val);
18     TParam(const TParam &oryginal);
19
20     std::string get_name() { return name; }
21
22     void set_range(double x_start, double x_end, double dx);
23
24     void set_rand_val();
25     void set_val(double val) { val_id = get_val_id(val); }
26     double get_val() { return x_start + val_id * dx; }
27
28     void info();
29
30 private:
31     int get_val_id(double val);
32     double get_x_start() { return x_start; }
33     double get_x_end() { return x_end; }
34     double get_dx() { return dx; }
35
36 };
37
```

Czasami zachodzi potrzeba utworzenia kopii obiektu. Najłatwiej ją utworzyć z użyciem konstruktora kopiującego (linia 18).

Implementujemy funkcje pomocnicze przy kopiowaniu wartości zmiennych obiektu.

Konstruktor kopiujący

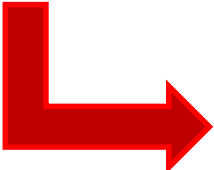
Idea kopiowania polega na skopiowaniu cech, tj. wartości zmiennych, istniejącego obiektu do nowego obiektu. **Kopiujemy => musimy więc obiecać, że nie zmienimy wartości istniejącego obiektu (podajemy go jako parametr stały).**

Konstruktor kopiujący

```
TParam.cpp  TCandidate.cpp  TPopulation.cpp  TAlgorithm.cpp
AG2024
30
31 TParam::TParam(const TParam &oryginal) : name{oryginal.get_name()}
32 {
33     double x_start = oryginal.get_x_start();
34     double x_end = oryginal.get_x_end();
35     double dx = oryginal.get_dx();
36     set_range(x_start, x_end, dx);
37
38     double val = oryginal.get_val();
39     set_val(val);
40 }
41
```

Konstruktor kopiujący

Idea kopiowania polega na skopiowaniu cech, tj. wartości zmiennych, istniejącego obiektu do nowego obiektu.



```
am.h  main.cpp  TCandidate.h  TPopulation
G2024 (Globalny zasięg)
1 #include <iostream>
2 #include <cstdlib> // srand
3 #include <time.h>
4
5 // #include "TAlgorithm.h"
6 #include "TParam.h"
7
8 using namespace std;
9
10
11 int main()
12 {
13     srand(time(0));
14
15     TParam gen{ "gen1", 0, 10, 0.1 };
16     gen.info();
17
18     TParam gen_copy(gen);
19     gen_copy.info();
20
21     std::cout << "\n\n\n";
22     return 0;
23 }
```

Microsoft Visual Studio

Wystąpiły błędy kompilacji. Czy chcesz kontynuować i uruchomić ostatnią kompilację zakończoną powodzeniem?

Tak

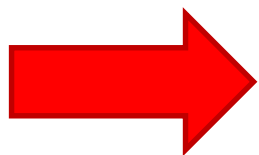
Nie

☐ Nie pokazuj tego okna dialogowego ponownie

Kopiujemy => musimy więc obiecać, że nie zmienimy wartości istniejącego obiektu.

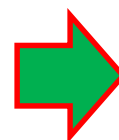
Konstruktor kopiujący

Kopiujemy => musimy więc obiecać, że nie zmienimy wartości istniejącego obiektu. (podajemy go jako parametr stały).



Skoro obiekt jest traktowany jako stała to wszystkie funkcje wywoływane na rzecz tego obiektu muszą obiecywać nie zmienianie stanu (wartości parametrów) tego obiektu.

```
6 class TParam
7 {
8 private:
9     const std::string name;
10    double x_start, x_end, dx;
11    int val_id;
12
13 public:
14    TParam(double x_start, double x_end, double dx);
15    TParam(double x_start, double x_end, double dx, double val);
16    TParam(std::string name, double x_start, double x_end, double dx);
17    TParam(std::string name, double x_start, double x_end, double dx, double val);
18    TParam(const TParam &original);
19
20    std::string get_name() const { return name; }
21    double get_val() const { return x_start + val_id * dx; }
22
23    void set_range(double x_start, double x_end, double dx);
24    void set_rand_val();
25    void set_val(double val) { val_id = get_val_id(val); }
26
27    void info();
28
29 private:
30    int get_val_id(double val);
31    double get_x_start() const { return x_start; }
32    double get_x_end() const { return x_end; }
33    double get_dx() const { return dx; }
34
35};
```



```
Konsola debugowania programu Microsoft Visual Studio

=====
== name: gen1
== range: [0; 10; 0.1]
== value: 9.3      (id: #93)
=====

=====
== name: gen1
== range: [0; 10; 0.1]
== value: 9.3      (id: #93)
=====
```

```
6 class TParam
7 {
8     private:
9         const std::string name;
10        double x_start, x_end, dx;
11        int val_id;
12
13    public:
14        TParam(double x_start, double x_end, double dx);
15        TParam(double x_start, double x_end, double dx, double val);
16        TParam(std::string name, double x_start, double x_end, double dx);
17        TParam(std::string name, double x_start, double x_end, double dx, double val);
18        TParam(const TParam &oryginal);
19
20        std::string get_name() const { return name; }
21        double get_val() const { return x_start + val_id * dx; }
22
23        void set_range(double x_start, double x_end, double dx);
24        void set_rand_val();
25        void set_val(double val) { val_id = get_val_id(val); }
26
27        void info();
28
29    private:
30        int get_val_id(double val);
31        double get_x_start() const { return x_start; }
32        double get_x_end() const { return x_end; }
33        double get_dx() const { return dx; }
34
35};
```

Deklarujemy funkcje
składowe jako stałe, czyli
takie które obiecują nie
zmieniać danych
składowych obiektu na
rzecz którego zostały
wywołane

Zadanie

Utworzyć konstruktory delegujące i kopiujące dla pozostałych klas projektu: *TCandidate* i ~~*TPopulation*~~.

```
main.h | main.cpp | TCandidate.h | TPopulation.h | TAlgorithm.h
G2024 | (Globalny zasięg)
1  #include <iostream>
2  #include <cstdlib> // srand
3  #include <time.h>
4
5  // #include "TAlgorithm.h"
6  #include "TParam.h"
7  #include "TCandidate.h"
8  #include "TPopulation.h"
9
10 using namespace std;
11
12
13 int main()
14 {
15     srand(time(0));
16
17     TParam gen{ "gen1", 0, 10, 0.1 };
18     gen.info();
19
20     TParam gen_copy(gen);
21     gen_copy.info();
22
23     // #####
24     std::cout << "\n\n";
25
26     TCandidate os1{};
27     os1.rate();
28     os1.info();
29
30     TCandidate os_copy{ os1 };
31     os_copy.info();
32
33     // #####
34     std::cout << "\n\n";
35
36     TPopulation pop1{ 5 };
37     pop1.calculate();
38     pop1.info();
39
40     TPopulation pop_copy{ pop1 };
41     pop_copy.info();
42
43     std::cout << "\n\n\n";
44     return 0;
45 }
46
```

Konsola debugowania programu Micr

```
=====
== name: gen1
== range: [0; 10; 0.1]
== value: 8.5 (id: #85)
=====

=====
== name: gen1
== range: [0; 10; 0.1]
== value: 8.5 (id: #85)
=====

=====
==
== gens count: 2
== "x1" value: 95
== "x2" value: 0
==
== rate: 9025
=====

=====
==
== gens count: 2
== "x1" value: 95
== "x2" value: 0
==
== rate: 9025
=====

===== POPULATION #0 =====
== candidate#0: 6249
== candidate#1: 3258
== candidate#2: 3372
== candidate#3: 9807
== candidate#4: 538
== best val: 9807
=====

liczba osobnikow: 5

===== POPULATION #1 =====
== candidate#0: 6249
== candidate#1: 3258
== candidate#2: 3372
== candidate#3: 9807
== candidate#4: 538
== best val: 9807
=====
```

Kopia genu

Kopia osobnika

Kopia populacji



Podpowiedź dot. konstruktora kopiującego populacji*



```
am.h  main.cpp  TCandidate.h  TPopulation.h  TAlgorithm.h
G2024  (Globalny zasięg)
1  #pragma once
2
3  #include <vector>
4  #include "TCandidate.h"
5
6  class TPopulation
7  {
8      static unsigned int    population_count;
9      unsigned int          _id;
10     unsigned int           candidate_count;
11     std::vector<TCandidate> candidates;
12     double                 best_val = 0;
13
14 public:
15     TPopulation(unsigned int cand_count = 10);
16
17     void calculate();
18     TCandidate get_best_candidate();
19
20     unsigned int get_id() { return _id; }
21     unsigned int get_candidates_count() const { return candidate_count; }
22     double get_best_val() const { return best_val; }
23
24     void info();
25
26 private:
27     const TCandidate* get_candidate_wsk(int _id) const;
28 };
29
```

```
78  const TCandidate* TPopulation::get_candidate_wsk(int _id) const
79  {
80      const TCandidate* wsk = &candidates[_id];
81      return wsk;
82  }
83
```

Fragment kodu konstruktora kopiującego

```
31  const TCandidate* wsk_os_org = original.get_candidate_wsk(i);
32  TCandidate copy{*wsk_os_org};
33  candidates.push_back(copy);
```